CS-781 Capstone, Phase 7: Prior Case Retrieval

Citation-window retrieval with the four Phase-1 language models

Manish Kumar (251330604)
CS-781, IIT Kanpur

November 2026

Abstract

For each query judgment we retrieve, from a split-specific pool of candidate judgments, the 7 prior cases it cites. Citations in the query text are masked as `<CITATION>`; the text around each marker describes the cited case, so both our lexical and our neural retrievers work per citation window. A bi-encoder on a Phase-1 backbone embeds each citation window and each 512-token chunk of a candidate and is trained contrastively with BM25 hard negatives; seven adaptation techniques (full and partial fine-tuning, LoRMA, QLoRA, IA^3, prefix and prompt tuning) and a distilled 4-block student cover all four architectures. The best neural model alone reaches micro-F1@7 0.389 on the development set; per-citation BM25 with a date penalty and pool-level normalisation reaches 0.482. The submitted system re-ranks the lexical candidates with LightGBM and adds the averaged scores of 13 trained Phase-1 retrievers: 0.499 on dev, against an upper bound of 0.618 for any 7-item list. All numbers are generated from the run outputs by `p7/report/make_figures.py`.

Contents

1 Introduction **2**

2 Data and its structure **2**

3 Methods **2**
3.1 Lexical retrieval . . . 2
3.2 Neural retriever on the Phase-1 models . . . 3
3.3 Adaptation techniques . . . 3
3.4 Re-ranking and combination . . . 4
3.5 Submissions . . . 4

4 Results **4**
4.1 Lexical retrieval . . . 4
4.2 Phase-1 models without training . . . 5
4.3 Adaptation experiments . . . 5
4.4 Knowledge distillation and efficiency . . . 6
4.5 Re-ranking and combination . . . 7
4.6 Errors of the submitted system . . . 7
4.7 Submission files . . . 7

5 Analysis **8**

6 Limitations and next steps **8**

7 Conclusions **9**

8 Reproducibility **9**

1 Introduction

Prior case retrieval (PCR) asks, for a query case, which candidate cases it relies on. Each query must receive exactly 7 distinct candidate ids from its own split's pool, ranked; the primary metric is micro-F1@7 (micro precision and recall over all queries, counting the first 7 ids against the full gold set), with micro precision and recall as secondary metrics. The rules require the Phase-1 models as the only pretrained weights (no other pretrained model, external corpora or LLM APIs), at least six substantially different experiments with at least five adaptation techniques, LoRMA and a knowledge-distillation experiment.

2 Data and its structure

Table 1: Splits. Self = queries that also occur in their own pool; Gold = relevant cases per query (mean, median, range); Cit. = median number of `<CITATION>` markers per query; lengths in words (median).

Split	Queries	Pool	Self	Gold mean	Gold med.	Gold range	Cit.	Query words	Case words
train	827	4,320	827	6.42	4	0–52	6	4,508	3,632
dev	118	1,023	118	8.25	5	1–92	7	4,620	4,036
test	237	1,727	237	–	–	–	7	4,653	3,947

Properties of the provided files that shape the method (checked before training):

- **Every query is in its own pool** and is never relevant to itself; it is always removed from its own ranking.
- **Citations are masked.** The number of `<CITATION>` markers in a query correlates 0.91 (dev) / 0.82 (train) with its number of gold cases.
- **A pool is essentially the union of its queries' gold cases plus the queries.** Of the pool cases that are not themselves queries, 99.9% (train) and 100.0% (dev) are gold for at least one query of the split; of the pool cases that are queries, 74.5% (train) but only 12.7% (dev) are.
- **Fixed list length.** Because every query gets exactly 7 ids while gold sets range from 1 to 92, even a perfect ranking scores micro-F1@7 0.618 on dev (0.639 on train).
- **Dates.** Judgment years are read from the `DATE OF JUDGMENT` header (or the latest year in the opening). With this extraction, 11% of dev gold cases appear dated after their query, so dates are used as a soft penalty, not a filter.
- **Overlap.** Pools of different splits share cases (740 test pool cases are also in the train pool); no relevance label crosses splits.

3 Methods

3.1 Lexical retrieval

`pcr/lexical.py`, scores cached by `lex.py` and `lex_wincol.py`. BM25 ($k_1 = 1.2$, $b = 0.75$) over word 1–3-grams (document frequency between 2 and 50% of the pool), with the pool as the collection.

- **Citation windows**: ±60 words around each `<CITATION>` of the query. *Per-citation* retrieval issues one BM25 query per window, z-scores each window's scores over the pool and takes the maximum over the query's windows.

- **Date penalty**: 2 standard deviations subtracted when the candidate's year is after the query's.
- **Column (case-wise) normalisation**: a query's z-scores are z-scored again across all queries of the split for each candidate, and added ("row + column"): a pool case is cited by some query, so it matters *which* query it fits best.
- **Window-level normalisation** (`per_citation_wincol`): the column step applied per window (each window's scores z-scored over cases, then each case over all windows of the split, row + γ column, $\gamma = 2$) before the maximum over windows.

3.2 Neural retriever on the Phase-1 models

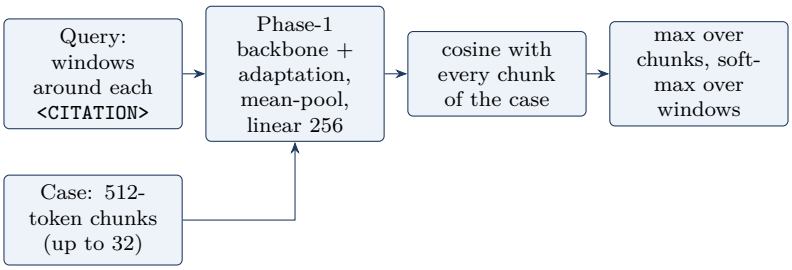


Figure 1: Citation-window bi-encoder (`pcr/model.py`).

- **Units**: a query is represented by its citation windows (±60 words, at most 256 tokens), a candidate by its 512-token chunks (32 evenly spread chunks at evaluation, 4 random chunks during training). Both pass through the same backbone, mean pooling and a 256-dimensional projection.
- **Score**: window–case similarity = maximum cosine over the case's chunks; query–case score = soft maximum over the query's windows (log-sum-exp, temperature 0.05).
- **Training**: 8 queries per step; for each, 2 gold cases and 3 hard negatives from the top 30 non-gold cases of per-citation BM25; every query is scored against every case in the batch with a multi-positive InfoNCE loss [8] (temperature 0.05, the query's own case removed). 10 epochs, AdamW, 5% warm-up then cosine decay; the epoch with the best dev micro-F1@7 is kept.
- **Backbones**: A1 with bidirectional attention over the window or chunk, A3 (bidirectional prefix), A4 causal, A2 decoder states over the window (encoder output collapsed, Phase-2 finding).

3.3 Adaptation techniques

- **Full fine-tuning** (E2; A1), learning rate 3×10^{-5}.
- **Partial fine-tuning** (E3; A4): the top 4 of 10 blocks, 10^{-4}.
- **LoRMA** [1] (E1; A3): $W = (I + \frac{\alpha}{r}BA)W_0$, $r = 32$, $\alpha = 64$, 5×10^{-4}.
- **QLoRA** [2] (E4; A2): frozen weights in 4-bit NF4, LoRA $r = 32$, 5×10^{-4}.
- **IA**^{3} [7] (E5; A3): learned rescaling of keys, values and the MLP, 3×10^{-3}.
- **Prefix tuning** [6] (E6; A1): 16 key/value positions per layer, 10^{-3}.
- **Prompt tuning** [5] (E7; A4): 20 input embeddings, 3×10^{-3}.

- **Knowledge distillation** [3] (K1): A1 student with 4 of its 8 blocks (0, 2, 5, 7), fully fine-tuned with $0.5\,\text{InfoNCE} + 0.5\,\text{KL}$ between the teacher's and the student's score distributions over the batch's cases (temperature 2τ); teacher = E2. K0 is the same student trained without the teacher.

Additional runs repeat E2, E1 and E3 with other seeds (E2b, E2c, E1b, E3b) and E2 with 6 hard negatives from the top 50, 6 chunks per case and 4 epochs (E2d).

3.4 Re-ranking and combination

- **Fused** (per run, `train.py`): z(neural) + λ z(per-citation BM25) − date penalty, λ and the penalty picked on dev.
- **Learning to rank** (`stack.py`, `pcr/ltr.py`): LightGBM lambdarank [4] over each query's top 100 candidates of the first stage. Features: row and column z-scores and ranks of every lexical score, date difference and violation, candidate length, number of citation windows, and whether the candidate is itself a query. Trained on train (3 seeds averaged), scored on dev; the test model is refit on train + dev.
- **Neural blend**: the ranker's z-scored output + $w\times$ the mean (row + column z-scored) score of several bi-encoder runs. Using a neural score as a ranker *feature* needs out-of-fold scores on train (2-fold runs F_E2_*); that variant (STK1) did not help.
- **Cross-encoder** (`cross_train.py`, C1): A1 reads [citation window, "Cited judgment:", case chunk] for the two best window–chunk pairs of each (query, case) of the STK4 top 30 and is trained listwise (1 gold against 11 non-gold).

3.5 Submissions

`prediction.json` maps every test query id to 7 distinct ids from the test pool, best first, never the query itself; every zip is checked against these rules when it is written.

4 Results

4.1 Lexical retrieval

Table 2: Lexical retrieval, micro-F1@7 (no learning).

Method	Dev	Train
BM25 on the whole query (word 1–3-grams)	0.3235	0.2525
BM25 on the citation contexts (±80 words)	0.3513	0.3062
per-citation BM25 (±60 words), max over windows	0.4325	0.3686
+ penalty for cases dated after the query	0.4525	0.3864
+ case-wise (column) normalisation	0.4691	0.4006
window-level normalisation (wc60, $\gamma = 2$) + column + date	0.4825	0.4057

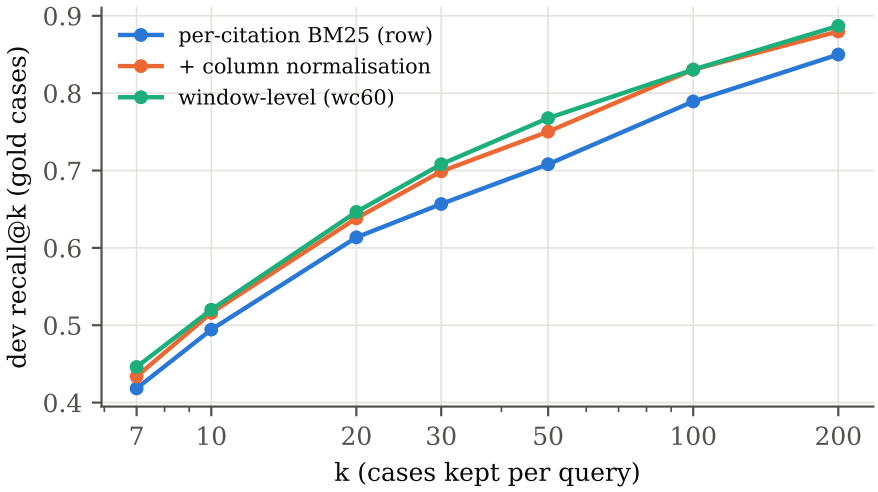


Figure 2: Share of dev gold cases among each query's top k lexical candidates (window-level first stage: 45% at $k = 7$, 83% at $k = 100$).

4.2 Phase-1 models without training

Table 3: Dev micro-F1@7 of the untrained Phase-1 backbones in the bi-encoder (mean-pooled states, no projection).

Backbone	Micro-F1@7
A1	0.2046
A2	0.2112
A3	0.1990
A4	0.2057

4.3 Adaptation experiments

Table 4: Dev results (118 queries). Neural = the bi-encoder alone (F1, P, R); Fused = with per-citation BM25 and the date penalty (weights picked on dev). Below the line: seeds and the hard-negative variant.

Run	Arch	Method	Total	Trainable	%	Neural F1	P	R	Fused F1
E1	A3	LoRMA	85.0M	3.74M	4.4	0.3791	0.413	0.350	0.4658
E2	A1	full fine-tuning	82.3M	82.26M	100.0	0.3891	0.424	0.360	0.4703
E3	A4	partial (top 4 blocks)	87.7M	25.37M	28.9	0.3835	0.418	0.355	0.4680
E4	A2	QLoRA (NF4)	26.2M	5.57M	21.3	0.3635	0.396	0.336	0.4636
E5	A3	IA^3	81.5M	0.23M	0.3	0.3669	0.400	0.339	0.4691
E6	A1	prefix tuning	89.0M	6.91M	7.8	0.3313	0.361	0.306	0.4658
E7	A4	prompt tuning	87.7M	0.21M	0.2	0.3357	0.366	0.310	0.4614
K1	A1	KD student, 4 blocks	53.9M	53.91M	100.0	0.3691	0.402	0.341	0.4625
K0	A1	4 blocks, no teacher	53.9M	53.91M	100.0	0.3535	0.385	0.327	0.4725
E2b	A1	full fine-tuning	82.3M	82.26M	100.0	0.3891	0.424	0.360	0.4703
E2c	A1	full fine-tuning	82.3M	82.26M	100.0	0.3802	0.414	0.351	0.4703
E2d	A1	full fine-tuning, 6 hard negatives	82.3M	82.26M	100.0	0.3747	0.408	0.346	0.4691
E1b	A3	LoRMA	85.0M	3.74M	4.4	0.3691	0.402	0.341	0.4680
E3b	A4	partial (top 4 blocks)	87.7M	25.37M	28.9	0.3691	0.402	0.341	0.4647

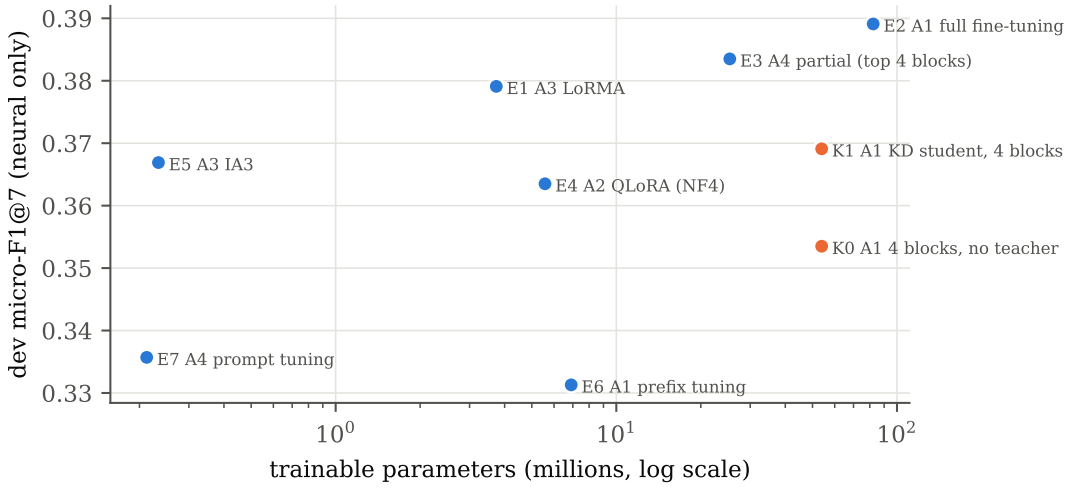


Figure 3: Neural-only dev micro-F1@7 against trained parameters (orange: 4-block students).

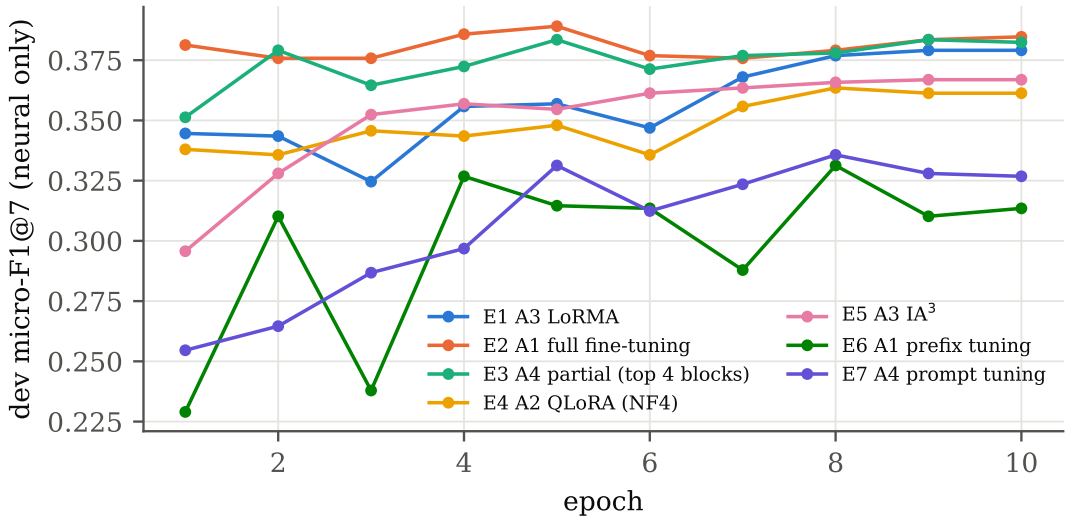


Figure 4: Neural-only dev micro-F1@7 after each epoch.

4.4 Knowledge distillation and efficiency

Table 5: Size, compute and memory. Encode = seconds to embed and score the whole dev split.

Run	Arch	Method	Params	Train (min)	Encode (s)	Peak GB	Ckpt MB
E1	A3	LoRMA	85.0M	18.5	11.4	35.6	15.0
E2	A1	full fine-tuning	82.3M	8.5	8.2	25.0	329.1
E3	A4	partial (top 4 blocks)	87.7M	15.7	22.8	17.8	101.5
E4	A2	QLoRA (NF4)	26.2M	21.7	17.3	46.8	22.3
E5	A3	IA^3	81.5M	15.4	14.6	22.5	0.9
E6	A1	prefix tuning	89.0M	12.6	16.9	15.1	27.6
E7	A4	prompt tuning	87.7M	19.4	6.6	25.9	0.9
K1	A1	KD student, 4 blocks	53.9M	8.1	1.6	13.9	215.7
K0	A1	4 blocks, no teacher	53.9M	6.2	2.2	12.9	215.7

K1 has 53.9M parameters against 82.3M for its teacher E2 and encodes the dev split in 1.6 s against 8.2 s; neural-only F1 0.369 (teacher 0.389, same student without teacher 0.353).

4.5 Re-ranking and combination

Table 6: Learning-to-rank stacks and the cross-encoder (dev). Runs = bi-encoders blended; w = blend weight. The C1 base (STK4 alone) scores 0.491; the cross-encoder alone re-orders the top 30 at 0.222.

Name	Description	Runs	w	F1	P	R
STK0_lex	lexical features only	0	–	0.4769	0.519	0.441
STK1_e2	+ E2 out-of-fold scores as features	0	–	0.4725	0.514	0.437
STK2_lex_blend4	lexical LTR + 4 bi-encoders blended (row z)	4	0.3	0.4869	0.530	0.450
STK4_rc_blend4	row+column features and first stage, + 4 bi-encoders (row+column)	4	0.5	0.4914	0.535	0.454
STK5_all13_w075	as STK4, 13 bi-encoders	13	0.75	0.4914	0.535	0.454
STK6_wc_all13	+ window-level lexical features and first stage, 13 bi-encoders	13	0.4	0.4992	0.544	0.462
C1 cross-encoder	STK4 top-30 re-ordered by z(STK4) + 0.25 z(cross)	–	–	0.4958	–	–

4.6 Errors of the submitted system

STK6 on dev: micro-F1@7 0.499 (P 0.544, R 0.461); 8 of 118 queries get no gold case.

Table 7: STK6 on dev by number of gold cases per query.

Gold cases	Queries	Hits in top 7	P	R	F1	Queries with 0 hits
1–3	40	1.45	0.207	0.806	0.330	7
4–6	30	3.53	0.505	0.716	0.592	1
7–10	20	5.10	0.729	0.637	0.680	0
11–20	19	6.37	0.910	0.467	0.617	0
21+	9	6.89	0.984	0.186	0.312	0

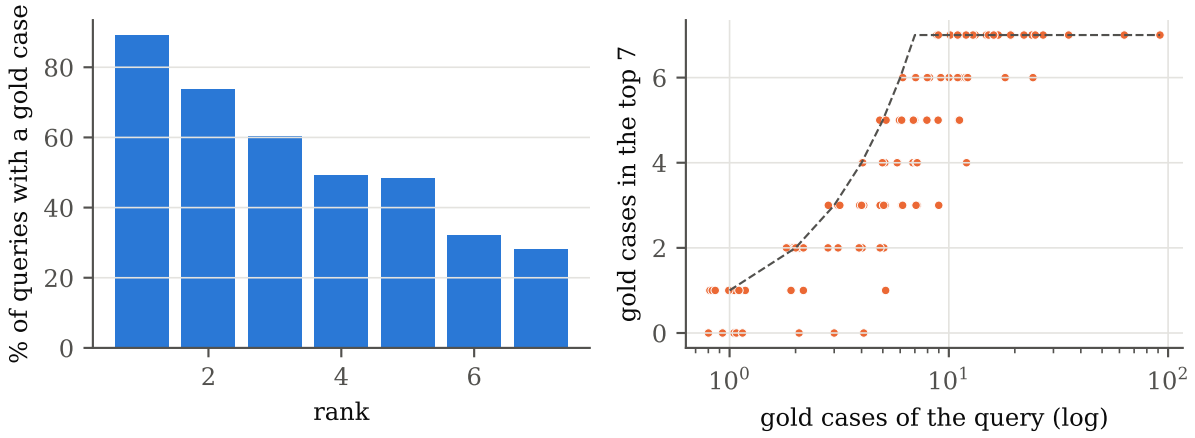


Figure 5: STK6 on dev. Left: share of queries with a gold case at each rank. Right: gold cases found in the top 7 against the query's number of gold cases (dashed: the maximum for small gold sets).

4.7 Submission files

35 format-checked zips are in `p7/submissions/`. Uploaded first: `STK6_wc_all13` (0.499), `STK5_all13_w075` and `STK4_rc_blend4` (0.491), then one zip per technique (`_neural`: Phase-1 model only; `_fused`: with BM25).

To be written by the author — leaderboard scores

Add the Codabench test micro-F1@7, precision and recall of each uploaded submission and say which six or more count towards the requirement.

File
C1_a1_cross_20261106_174650.zip
E1_a3_lorma_fused_20261104_030307.zip
E1_a3_lorma_neural_20261104_030306.zip
E1b_a3_lorma_seed1_fused_20261106_180415.zip
E1b_a3_lorma_seed1_neural_20261106_180415.zip
E2_a1_full_fused_20261104_031321.zip
E2_a1_full_neural_20261104_031321.zip
E2b_a1_full_seed1_fused_20261106_181453.zip
E2b_a1_full_seed1_neural_20261106_181453.zip
E2c_a1_full_seed2_fused_20261106_181510.zip
E2c_a1_full_seed2_neural_20261106_181510.zip
E2d_a1_full_hard_fused_20261106_182019.zip
E2d_a1_full_hard_neural_20261106_182019.zip
E3_a4_partial_fused_20261104_032116.zip
E3_a4_partial_neural_20261104_032116.zip
E3b_a4_partial_seed1_fused_20261106_180635.zip
E3b_a4_partial_seed1_neural_20261106_180635.zip
E4_a2_qlora_fused_20261104_032709.zip
E4_a2_qlora_neural_20261104_032709.zip
E5_a3_ia3_fused_20261104_030010.zip
E5_a3_ia3_neural_20261104_030009.zip
E6_a1_prefix_fused_20261104_025721.zip
E6_a1_prefix_neural_20261104_025721.zip
E7_a4_prompt_fused_20261104_030353.zip
E7_a4_prompt_neural_20261104_030353.zip
K0_a1_4l_nokd_fused_20261104_033149.zip
K0_a1_4l_nokd_neural_20261104_033149.zip
K1_kd_a1_4l_fused_20261104_033342.zip
K1_kd_a1_4l_neural_20261104_033342.zip
STK0_lex_20261104_031622.zip
STK1_e2_20261104_032324.zip
STK2_lex_blend4_20261104_032450.zip
STK4_rc_blend4_20261104_033342.zip
STK5_all13_w075_20261106_182110.zip
STK6_wc_all13_20261106_192849.zip

5 Analysis

To be written by the author — analysis of the results

The course rules do not allow LLM-written result analysis. Questions the course asks (§13.5), with the evidence above:

- How do the adaptation techniques affect retrieval quality (Table 4, Figure 3)?
- How do the Phase-1 representations transfer to PCR (Table 3 vs. Table 4)?
- What is the effect of pooling and document representation (citation windows, chunk maximum, Table 2)?
- What is the effect of the similarity and ranking objectives (InfoNCE, learning to rank, blending, Table 6)?
- What do hard negatives and negative sampling change (E2 vs. E2d)?
- Which challenges do long legal documents pose?
- What are the common retrieval errors (Table 7, Figure 5)?
- How effective is knowledge distillation (K1 vs. K0 and E2, Table 5)?
- What is the trade-off between quality, size, memory and inference time?
- Why might a technique that works on another legal task not transfer to PCR?

6 Limitations and next steps

Facts that limit the conclusions (the interpretation is the author's):

- Dev has 118 queries; differences of about 0.015 micro-F1@7 are within noise. Blend weights and stack variants were chosen on dev (the ranker's own settings were checked by 5-fold cross-validation on train).
- The submitted stacks use BM25 and the pool structure (Section 2); neither uses a pretrained model, outside data or test labels, but they are not Phase-1 models. The `_neural` zips use the Phase-1 models only.
- One seed for most runs; E2/E2b/E2c differ by up to 0.009.
- Judgment years are extracted heuristically (Section 2).

To be written by the author — what you would do next
With more time or compute: ...

7 Conclusions

To be written by the author — conclusions
Summarise what you conclude from the experiments, in your own words.

8 Reproducibility

Lexical scores	`python lex.py`; `python lex_wincol.py`
Token cache	`pcr/tok.py` (`build`)
Experiments	`P=2 bash run_queue.sh e1_a3_lorma e2_a1_full ... k1_kd_a1_4l k0_a1_4l_nokd`
One experiment	`python train.py -config configs/e2_a1_full.yaml`
Out-of-fold runs	`bash folds.sh e2_a1_full`
Final stack	`python stack.py -name STK6_wc_all13 -first wc60_g2 -blend E2_a1_full ... K1_kd_a1_4l -w 0.4 -refit`
Cross-encoder	`python cross_prep.py`; `python cross_train.py -config configs/c1_a1_cross.yaml`
Figures and tables	`python p7/report/make_figures.py`

To be written by the author — repository link
GitHub: `https://github.com/...`

LLM declaration

To be written by the author — LLM declaration (mandatory)
Fill in the course questionnaire. Describe honestly which parts used an LLM assistant (for example the data pipeline, model and training code, figures and this report's methods/results text) and which parts are your own (design decisions, hyperparameters, analysis, conclusions).

References

[1] H. Bihany, S. Patel, A. Modi. LoRMA: Low-rank multiplicative adaptation for LLMs. Findings of ACL 2025.

[2] T. Dettmers et al. QLoRA: Efficient finetuning of quantized LLMs. NeurIPS 2023.

[3] G. Hinton, O. Vinyals, J. Dean. Distilling the knowledge in a neural network. arXiv:1503.02531, 2015.

[4] G. Ke et al. LightGBM: A highly efficient gradient boosting decision tree. NeurIPS 2017.

[5] B. Lester, R. Al-Rfou, N. Constant. The power of scale for parameter-efficient prompt tuning. EMNLP 2021.

[6] X. L. Li, P. Liang. Prefix-tuning: Optimizing continuous prompts for generation. ACL 2021.

[7] H. Liu et al. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. NeurIPS 2022.

[8] A. van den Oord, Y. Li, O. Vinyals. Representation learning with contrastive predictive coding. arXiv:1807.03748, 2018.